ASSIGNMENT-2 CO3

1. HEADER

Name:	B Baby Suharshitha
Roll No.:	05
Course Code & CO Reference:	CSA0920, CO3
Assignment Set & Question No.:	Direct-Descriptive, Q05
GitHub Repository Link:	https://github.com/suharshithabathina/CSA0920-JAVA-
Submission Date:	11-08-2026

Assigned Question Text:

Nested Try-Catch Blocks

Write a Java program that contains a try block nested inside another try block. The outer try should handle a general exception, while the inner try handles a more specific exception related to a sub-operation (for example, an outer block managing overall program flow and an inner block specifically validating a numeric conversion). Demonstrate how control passes between the nested blocks when an exception occurs at the inner level.

Concepts: nested try-catch.

2. PROBLEM UNDERSTANDING

The objective of this program is to demonstrate hierarchical exception handling using **nested try-catch blocks** in Java.

When executing complex applications, specific sub-tasks (such as string-to-numeric conversion) can fail independently of higher-level operations (such as mathematical computations or overall flow management).

- **Inner Try-Catch Block:** Designed to intercept granular, localized exceptions (e.g., NumberFormatException) that occur during sub-operations. Handled locally, it allows recovery or default value assignment without breaking the outer program flow.
- **Outer Try-Catch Block:** Acts as a safety net for broader, program-level exceptions (e.g., ArithmeticException or generic Exception).

Demonstrating this control flow highlights how Java searches sequentially for matching handlers from the innermost block outward.

3. APPROACH / DESIGN NOTES

- **Class Selection:** Used java.util.Scanner for interactive terminal input.

- **Outer Scope Management:** Wraps high-level workflow, specifically performing division arithmetic, catching runtime errors like division by zero (ArithmeticException) or unhandled generic errors (Exception).
- **Inner Scope Management:** Encapsulates numeric parsing logic (Integer.parseInt()). If non-numeric strings are provided, the inner catch block traps NumberFormatException, sets a safe fallback default, and prevents early program termination.
- **Control Flow Mechanics:**

1. If the inner try block throws NumberFormatException, control passes immediately to the **inner catch block**.
2. Once the inner catch block completes execution, control continues with the remaining statements inside the **outer try block**.

3. If an uncaught exception occurs or an error happens directly in the outer scope (e.g., 100 / 0), control bypasses remaining logic and jumps directly to the **outer catch block**.

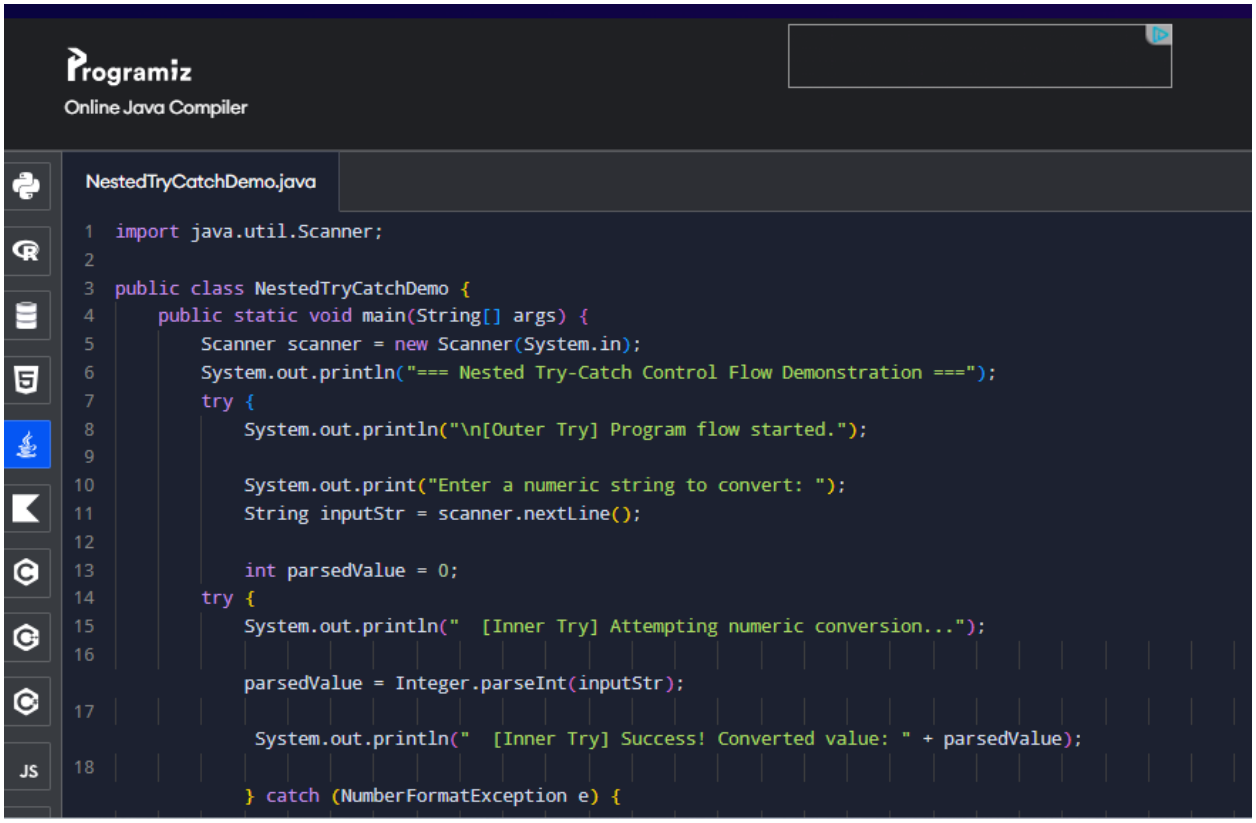
- **Rejected Alternative:** Sequential non-nested try-catch blocks. Sequential blocks do not express sub-task context or hierarchical dependencies where an outer block relies on localized recovery from an inner block.

4. SOURCE CODE

```
import java.util.Scanner;

public class NestedTryCatchDemo {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.println("=== Nested Try-Catch Control Flow Demonstration ===");

            try {

            System.out.println("\n[Outer Try] Program flow started.");

            System.out.print("Enter a numeric string to convert: ");

            String inputStr = scanner.nextLine();

            int parsedValue = 0;

            try {

                System.out.println("  [Inner Try] Attempting numeric conversion...");

                parsedValue = Integer.parseInt(inputStr);
```

```
            System.out.println("    [Inner Try] Success! Converted value: " + parsedValue);

        } catch (NumberFormatException e) {

            System.out.println("    [Inner Catch] Intercepted specific NumberFormatException!");

            System.out.println("  [Inner Catch] Input was non-numeric. Assigning default fallback value = 10.");

            parsedValue = 10; // Graceful recovery

        }

        System.out.println("[Outer Try] Proceeding to mathematical operation using value: " + parsedValue);

        System.out.print("Enter an integer divisor: ");

        int divisor = Integer.parseInt(scanner.nextLine());

        int result = 100 / divisor; // Potential ArithmeticException (division by zero)

        System.out.println("[Outer Try] Calculation successful: 100 / " + divisor + " = " + result);

    } catch (ArithmeticException e) {

        System.out.println("[Outer Catch] Handled ArithmeticException: Division by zero is not allowed!");

    } catch (Exception e) {

        System.out.println("[Outer Catch] Handled General Exception: " + e.getMessage());
```

```
        } finally {
            System.out.println("\n[Finally Block] Cleanup executed. Program terminating cleanly.");
            scanner.close();
        }
    }
}
```

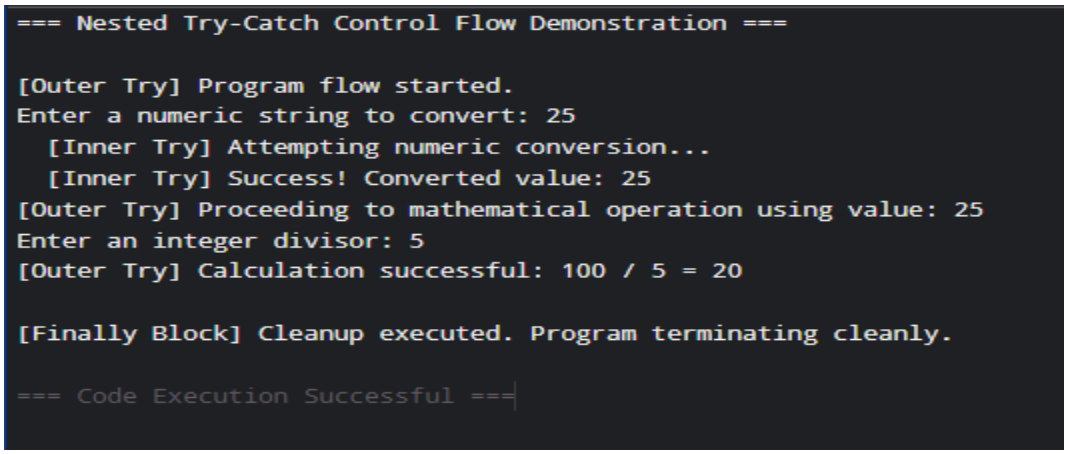


5. TEST CASES

s.no.	Input	Expected Output	Actual Output
1	**Numeric String:** 25 **Divisor:** 5 *(Typical Case)*	Inner conversion successful (25). Outer division result (100 / 5 = 20). Normal completion.	Inner conversion successful (25). Outer division result (100 / 5 = 20). Normal completion.
2	**Numeric String:** abc **Divisor:** 4 *(Inner Exception Case)*	Inner catch traps NumberFormatException, sets fallback (10). Outer division continues (100 / 4 = 25).	Inner catch traps NumberFormatException, sets fallback (10). Outer division continues (100 / 4 = 25).
3	**Numeric String:** 50 **Divisor:** 0 *(Outer Exception Case)*	Inner conversion successful (50). Outer try throws division by zero caught by Outer ArithmeticException handler.	Inner conversion successful (50). Outer try throws division by zero caught by Outer ArithmeticException handler.

6. SCREENSHOTS OF EXECUTION

Test Case 1 (Typical case)



Test Case 2 (Inner Exception Case)

```
=== Nested Try-Catch Control Flow Demonstration ===

[Outer Try] Program flow started.
Enter a numeric string to convert: abc
  [Inner Try] Attempting numeric conversion...
  [Inner Catch] Intercepted specific NumberFormatException!
  [Inner Catch] Input was non-numeric. Assigning default fallback value = 10.
[Outer Try] Proceeding to mathematical operation using value: 10
Enter an integer divisor: 4
[Outer Try] Calculation successful: 100 / 4 = 25

[Finally Block] Cleanup executed. Program terminating cleanly.

=== Code Execution Successful ===
```

Test Case 3 (Outer Exception Case)

```
=== Nested Try-Catch Control Flow Demonstration ===

[Outer Try] Program flow started.
Enter a numeric string to convert: 50
  [Inner Try] Attempting numeric conversion...
  [Inner Try] Success! Converted value: 50
[Outer Try] Proceeding to mathematical operation using value: 50
Enter an integer divisor: 0
[Outer Catch] Handled ArithmeticException: Division by zero is not allowed!

[Finally Block] Cleanup executed. Program terminating cleanly.

=== Code Execution Successful ===
```

7. REFLECTION

Through this assignment, I deepened my understanding of Java's call-stack resolution during exception propagation in nested structures:

- Localized recovery strategies allow granular sub-tasks to fail safely without terminating broader program operations.
- When an inner exception matching an inner catch block occurs, control flow transfers immediately to that inner catch block, resumes execution immediately after the inner block, and stays inside the outer try block context.
- If an exception in the inner block does not match any local handler, Java propagates the exception up to the enclosing outer catch blocks.

8. DECLARATION

"This is my own work. Any AI/external tool assistance used is disclosed above. No external assistance was used in writing the core code logic."